

# **VISVESVARAYA TECHNOLOGICAL UNIVERSITY**

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT  
on

## **OPERATING SYSTEMS**

Submitted by

VEMULA VENKATA JAIDHEER(1BF24CS333)

in partial fulfilment for the award of the degree of  
**BACHELOR OF ENGINEERING**  
in  
**COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING**

(Autonomous Institution under VTU)

**BENGALURU-560019**

**Feb-2026 to June-2026**

B. M. S. College of Engineering,  
Bull Temple Road, Bangalore 560019  
(Affiliated To Visvesvaraya Technological University, Belgaum)  
Department of Computer Science and Engineering



**CERTIFICATE**

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Vemula Venkata Jaidheer(1BF24CS333), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2024. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil  
Associate Professor  
Department of CSE  
BMSCE, Bengaluru

Dr. Kavitha Sooda  
Professor and Head  
Department of CSE  
BMSCE, Bengaluru

| <b>Sl. No.</b> | <b>Experiment Title</b>                                                                                                                                                                                                                                                                                                                    | <b>Page No.</b> |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| 1.             | Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.<br>a) FCFS<br>b) SJF<br>c) Priority<br>d) Round Robin (Experiment with different quantum sizes for RR algorithm)                                                                                                            | 3               |
| 2.             | Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue. | 20              |
| 3.             | Write a C program to simulate Real-Time CPU Scheduling algorithms:<br>a) Rate- Monotonic<br>b) Earliest-deadline First<br>c) Proportional scheduling                                                                                                                                                                                       | 23              |
| 4.             | Write a C program to simulate:<br>a) Producer-Consumer problem using semaphores.<br>b) Dining-Philosopher's problem                                                                                                                                                                                                                        | 32              |
| 5.             | Write a C program to simulate:<br>a) Bankers' algorithm for the purpose of deadlock avoidance.<br>b) Deadlock Detection                                                                                                                                                                                                                    | 38              |
| 6.             | Write a C program to simulate the following contiguous memory allocation techniques.<br>a) Worst-fit<br>b) Best-fit<br>c) First-fit                                                                                                                                                                                                        | 46              |
| 7.             | Write a C program to simulate page replacement algorithms.<br>a) FIFO<br>b) LRU<br>c) Optimal                                                                                                                                                                                                                                              | 50              |

1. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

**a) FCFS**

```
#include <stdio.h>

int main()
{
    int n, i;

    printf("Enter the number of processes:");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], wt[n], tat[n];

    for(i = 0; i < n; i++)
    {
        printf("Enter Arrival Time for Process %d:", i + 1);
        scanf("%d", &at[i]);

        printf("Enter Burst Time for Process %d:", i + 1);
        scanf("%d", &bt[i]);
    }

    ct[0] = at[0] + bt[0];

    for(i = 1; i < n; i++)
    {
        if(ct[i - 1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i - 1] + bt[i];
    }

    float totalWT = 0, totalTAT = 0;

    for(i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        totalWT += wt[i];
        totalTAT += tat[i];
    }
```

```

printf("\nPROCESS\tAT\tBT\tCT\tWT\tTAT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", totalWT / n);
printf("\nAverage Turnaround Time = %.2f\n", totalTAT / n);

return 0;
}

```

### **OUTPUT:**

```

C:\Users\BMSCECSE-L3-47\De  X  +  v
Enter the number of processes:4
Enter Arrival Time for Process 1:0
Enter Burst Time for Process 1:7
Enter Arrival Time for Process 2:0
Enter Burst Time for Process 2:3
Enter Arrival Time for Process 3:0
Enter Burst Time for Process 3:4
Enter Arrival Time for Process 4:0
Enter Burst Time for Process 4:6

PROCESS AT      BT      CT      WT      TAT
P1      0      7      7      0      7
P2      0      3      10     7      10
P3      0      4      14     10     14
P4      0      6      20     14     20

Average Waiting Time = 7.75
Average Turnaround Time = 12.75

Process returned 0 (0x0)   execution time : 40.811 s
Press any key to continue.
|

```

## **b) SJF (pre-emptive)**

```
#include<stdio.h>

struct process{
    int pid, bt, at, wt, tat, ct;
};

int main(){
    struct process p[10], temp;
    int n,i,j;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("Process %d Arrival Time: ",i+1);
        scanf("%d",&p[i].at);
        printf("Process %d Burst Time: ",i+1);
        scanf("%d",&p[i].bt);
        p[i].pid=i+1;
    }

    for(i=0;i<n-1;i++){
        for(j=i+1;j<n;j++){
            if(p[i].bt > p[j].bt){
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }

    int time=0;
    for(i=0;i<n;i++){
        time += p[i].bt;
        p[i].ct=time;
        p[i].tat=p[i].ct-p[i].at;
        p[i].wt=p[i].tat-p[i].bt;
    }

    float avgwt=0,avgtat=0;

    printf("\nPID\tAT\tBT\tWT\tTAT\n");
```

```

for(i=0;i<n;i++){
    printf("%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].wt,p[i].tat);
    avgwt+=p[i].wt;
    avgtat+=p[i].tat;
}

printf("\nAverage Waiting Time = %.2f",avgwt/n);
printf("\nAverage Turnaround Time = %.2f",avgtat/n);
}

```

### **OUTPUT:**

```

C:\Users\BMSCECSE-L3-43\De X + v
Enter number of processes: 4
Process 1 Arrival Time: 0
Process 1 Burst Time: 7
Process 2 Arrival Time: 0
Process 2 Burst Time: 3
Process 3 Arrival Time: 0
Process 3 Burst Time: 4
Process 4 Arrival Time: 0
Process 4 Burst Time: 6

PID    AT    BT    WT    TAT
2       0     3     0     3
3       0     4     3     7
4       0     6     7    13
1       0     7    13    20

Average Waiting Time = 5.75
Average Turnaround Time = 10.75
Process returned 0 (0x0)   execution time : 27.624 s
Press any key to continue.
|

```

```
C:\Users\BMSCECSE-L3-43\De X + v
Enter number of processes: 3
Process 1 Arrival Time: 0
Process 1 Burst Time: 8
Process 2 Arrival Time: 1
Process 2 Burst Time: 6
Process 3 Arrival Time: 2
Process 3 Burst Time: 7

PID    AT    BT    WT    TAT
2       1     6     -1     5
3       2     7     4     11
1       0     8     13     21

Average Waiting Time = 5.33
Average Turnaround Time = 12.33
Process returned 0 (0x0)  execution time : 10.913 s
Press any key to continue.
|
```



### **b) SJF (non pre-emptive)**

```
#include<stdio.h>

int main(){
    int bt[10],at[10],rt[10],ct[10],tat[10],wt[10];
    int n,i,time=0,remain,smallest;
    float avgwt=0,avgtat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("Arrival Time: ");
        scanf("%d",&at[i]);
        printf("Burst Time: ");
        scanf("%d",&bt[i]);
        rt[i]=bt[i];

    }
    remain=n;

    while(remain!=0){
        smallest=-1;

        for(i=0;i<n;i++){
            if(at[i]<=time && rt[i]>0){
                if(smallest==-1 || rt[i]<rt[smallest])
                    smallest=i;
            }
        }

        if(smallest==-1){
            time++;
            continue;
        }

        rt[smallest]--;
        time++;

        if(rt[smallest]==0){
            remain--;
            ct[smallest]=time;
            tat[smallest]=ct[smallest]-at[smallest];
```

```

        wt[smallest]=tat[smallest]-bt[smallest];
    }
}

printf("\nPID\tAT\tBT\tWT\tTAT\n");

for(i=0;i<n;i++){
    printf("%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],wt[i],tat[i]);
    avgwt+=wt[i];
    avgtat+=tat[i];
}

printf("\nAverage WT=%.2f",avgwt/n);
printf("\nAverage TAT=%.2f",avgtat/n);
}

```

### **OUTPUT:**

```

C:\Users\BMSCECSE-L3-43\De...
Enter number of processes: 4
Arrival Time: 0
Burst Time: 7
Arrival Time: 0
Burst Time: 3
Arrival Time: 0
Burst Time: 4
Arrival Time: 0
Burst Time: 6

PID    AT    BT    WT    TAT
1       0     7    13    20
2       0     3     0     3
3       0     4     3     7
4       0     6     7    13

Average WT=5.75
Average TAT=10.75
Process returned 0 (0x0)   execution time : 35.209 s
Press any key to continue.
|

```

```
C:\Users\BMSCECSE-L3-43\De X + v
Enter number of processes: 3
Arrival Time: 0
Burst Time: 8
Arrival Time: 1
Burst Time: 6
Arrival Time: 2
Burst Time: 7

PID    AT    BT    WT    TAT
1       0     8     6     14
2       1     6     0     6
3       2     7    12    19

Average WT=6.00
Average TAT=13.00
Process returned 0 (0x0)  execution time : 12.869 s
Press any key to continue.
```

### c) Priority (pre emptive)

```
#include<stdio.h>

struct process{
    int pid,bt,at,pr,wt,tat,ct;
};

int main(){
    struct process p[10],temp;
    int n,i,j;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("Arrival Time: ");
        scanf("%d",&p[i].at);

        printf("Burst Time: ");
        scanf("%d",&p[i].bt);

        printf("Priority: ");
        scanf("%d",&p[i].pr);

        p[i].pid=i+1;
    }

    for(i=0;i<n-1;i++){
        for(j=i+1;j<n;j++){
            if(p[i].pr > p[j].pr){
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }

    int time=0;
    float avgwt=0,avgtat=0;

    for(i=0;i<n;i++){
        time+=p[i].bt;
        p[i].ct=time;
```

```

        p[i].tat=p[i].ct-p[i].at;
        p[i].wt=p[i].tat-p[i].bt;
    }

    printf("\nPID\tPR\tWT\tTAT\n");

    for(i=0;i<n;i++){
        printf("%d\t%d\t%d\t%d\n",p[i].pid,p[i].pr,p[i].wt,p[i].tat);
        avgwt+=p[i].wt;
        avgtat+=p[i].tat;
    }

    printf("\nAverage WT=%.2f",avgwt/n);
    printf("\nAverage TAT=%.2f",avgtat/n);
}

```

## **OUTPUT:**

```

C:\Users\BMSCECSE-L3-43\Desktop >
Enter number of processes: 4
Arrival Time: 0
Burst Time: 7
Priority: 1
Arrival Time: 0
Burst Time: 3
Priority: 2
Arrival Time: 0
Burst Time: 4
Priority: 3
Arrival Time: 0
Burst Time: 6
Priority: 4

PID    PR    WT    TAT
1      1     0     7
2      2     7    10
3      3    10    14
4      4    14    20

Average WT=7.75
Average TAT=12.75
Process returned 0 (0x0)   execution time : 42.038 s
Press any key to continue.
|

```

```
C:\Users\BMSCECSE-L3-43\De X + v
Enter number of processes: 3
Arrival Time: 0
Burst Time: 8
Priority: 2
Arrival Time: 1
Burst Time: 6
Priority: 1
Arrival Time: 2
Burst Time: 7
Priority: 3

PID    PR    WT    TAT
2       1     -1     5
1       2      6    14
3       3     12    19

Average WT=5.67
Average TAT=12.67
Process returned 0 (0x0)   execution time : 47.356 s
Press any key to continue.
|
```

### c) Priority (Non pre-emptive)

```
#include<stdio.h>

int main(){
    int bt[10],at[10],pr[10],rt[10];
    int wt[10],tat[10],ct[10];
    int n,time=0,remain,highest,i;

    float avgwt=0,avgtat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("Arrival Time: ");
        scanf("%d",&at[i]);

        printf("Burst Time: ");
        scanf("%d",&bt[i]);

        printf("Priority: ");
        scanf("%d",&pr[i]);

        rt[i]=bt[i];
    }

    remain=n;

    while(remain!=0){
        highest=-1;

        for(i=0;i<n;i++){
            if(at[i]<=time && rt[i]>0){
                if(highest==-1 || pr[i]<pr[highest])
                    highest=i;
            }
        }

        if(highest!=-1){
            time++;
            continue;
        }
    }
```

```

rt[highest]--;
time++;

if(rt[highest]==0){
    remain--;
    ct[highest]=time;
    tat[highest]=ct[highest]-at[highest];
    wt[highest]=tat[highest]-bt[highest];
}
}

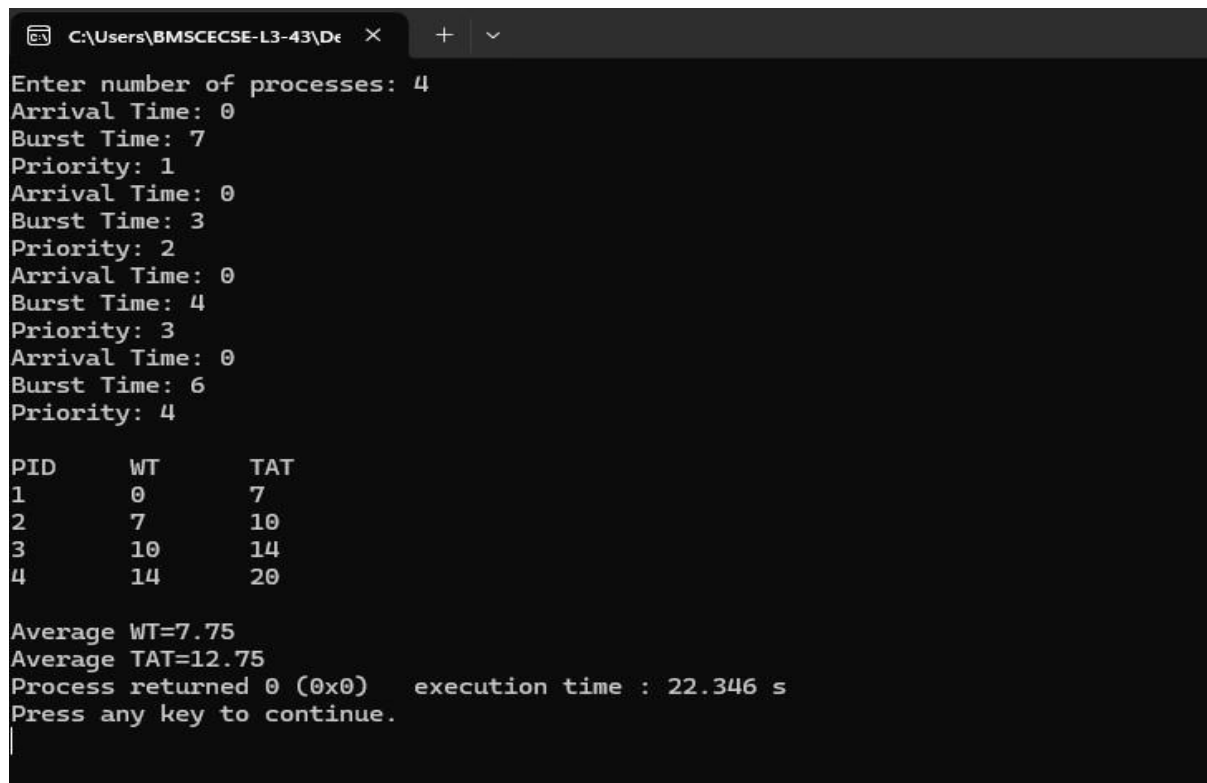
printf("\nPID\tWT\tTAT\n");

for(i=0;i<n;i++){
    printf("%d\t%d\t%d\n",i+1,wt[i],tat[i]);
    avgwt+=wt[i];
    avgtat+=tat[i];
}

printf("\nAverage WT=%.2f",avgwt/n);
printf("\nAverage TAT=%.2f",avgtat/n);
}

```

### **OUTPUT:**



```

C:\Users\BMSCECSE-L3-43\De
Enter number of processes: 4
Arrival Time: 0
Burst Time: 7
Priority: 1
Arrival Time: 0
Burst Time: 3
Priority: 2
Arrival Time: 0
Burst Time: 4
Priority: 3
Arrival Time: 0
Burst Time: 6
Priority: 4

PID      WT      TAT
1         0       7
2         7      10
3        10      14
4        14      20

Average WT=7.75
Average TAT=12.75
Process returned 0 (0x0)   execution time : 22.346 s
Press any key to continue.

```



```
C:\Users\BMSCECSE-L3-43\De X + v
Enter number of processes: 3
Arrival Time: 0
Burst Time: 8
Priority: 2
Arrival Time: 1
Burst Time: 6
Priority: 1
Arrival Time: 2
Burst Time: 7
Priority: 3

PID    WT    TAT
1       6     14
2       0      6
3      12     19

Average WT=6.00
Average TAT=13.00
Process returned 0 (0x0)  execution time : 25.908 s
Press any key to continue.
|
```

#### **d) Round Robin**

```
#include <stdio.h>

int main()
{
    int n, i, tq;
    int at[20], bt[20], rem_bt[20];
    int ct[20], wt[20], tat[20];
    int time = 0, done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Arrival Time and Burst Time for each process:\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d Arrival Time: ", i+1);
        scanf("%d", &at[i]);

        printf("P%d Burst Time: ", i+1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    do
    {
        done = 1;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= time && rem_bt[i] > 0)
            {
                done = 0;

                if(rem_bt[i] > tq)
                {
                    time += tq;
                    rem_bt[i] -= tq;
                }
            }
        }
    }
    while(done == 0);
}
```

```

        }
    else
    {
        time += rem_bt[i];
        ct[i] = time;
        rem_bt[i] = 0;
    }
}

if(done == 1)
    time++;

} while(done == 0);

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

float avg_wt = 0, avg_tat = 0;

printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], wt[i], tat[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);

return 0;
}

```

```
C:\Users\BMSCECSE-L3-43\De X + v
Enter number of processes: 3
Enter Arrival Time and Burst Time for each process:
P1 Arrival Time: 2
P1 Burst Time: 5
P2 Arrival Time: 1
P2 Burst Time: 3
P3 Arrival Time: 0
P3 Burst Time: 1
Enter Time Quantum: 3

Process AT      BT      CT      WT      TAT
P1      2      5      9      2      7
P2      1      3      4      0      3
P3      0      1      1      0      1

Average Waiting Time = 0.67
Average Turnaround Time = 3.67

Process returned 0 (0x0)  execution time : 18.898 s
Press any key to continue.
```

```
C:\Users\BMSCECSE-L3-43\De X + v
Enter number of processes: 3
Enter Arrival Time and Burst Time for each process:
P1 Arrival Time: 2
P1 Burst Time: 5
P2 Arrival Time: 1
P2 Burst Time: 3
P3 Arrival Time: 0
P3 Burst Time: 1
Enter Time Quantum: 2

Process AT      BT      CT      WT      TAT
P1      2      5      9      2      7
P2      1      3      6      2      5
P3      0      1      1      0      1

Average Waiting Time = 1.33
Average Turnaround Time = 4.33

Process returned 0 (0x0)  execution time : 14.021 s
Press any key to continue.
```

2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>

#define MAX 100

int main() {
    int n, i, j;

    int PID[MAX], AT[MAX], BT[MAX], TYPE[MAX];
    int CT[MAX], TAT[MAX], WT[MAX], Start[MAX];

    int sys[MAX], user[MAX];
    int sysCount = 0, userCount = 0;

    int time = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("PID: ");
        scanf("%d", &PID[i]);
        printf("Arrival Time: ");
        scanf("%d", &AT[i]);
        printf("Burst Time: ");
        scanf("%d", &BT[i]);
        printf("Type (0 = System, 1 = User): ");
        scanf("%d", &TYPE[i]);
    }

    for(i = 0; i < n; i++) {
        if(TYPE[i] == 0)
            sys[sysCount++] = i;
        else
            user[userCount++] = i;
    }

    for(i = 0; i < sysCount - 1; i++) {
        for(j = i + 1; j < sysCount; j++) {
            if(AT[sys[i]] > AT[sys[j]]) {
                int temp = sys[i];
```

```

        sys[i] = sys[j];
        sys[j] = temp;
    }
}

for(i = 0; i < userCount - 1; i++) {
    for(j = i + 1; j < userCount; j++) {
        if(AT[user[i]] > AT[user[j]]) {
            int temp = user[i];
            user[i] = user[j];
            user[j] = temp;
        }
    }
}

for(i = 0; i < sysCount; i++) {
    int p = sys[i];

    if(time < AT[p])
        time = AT[p];

    Start[p] = time;
    time += BT[p];
    CT[p] = time;

    TAT[p] = CT[p] - AT[p];
    WT[p] = TAT[p] - BT[p];
}

for(i = 0; i < userCount; i++) {
    int p = user[i];

    if(time < AT[p])
        time = AT[p];

    Start[p] = time;
    time += BT[p];
    CT[p] = time;

    TAT[p] = CT[p] - AT[p];
    WT[p] = TAT[p] - BT[p];
}

printf("\nPID\tAT\tBT\tTYPE\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++) {

```

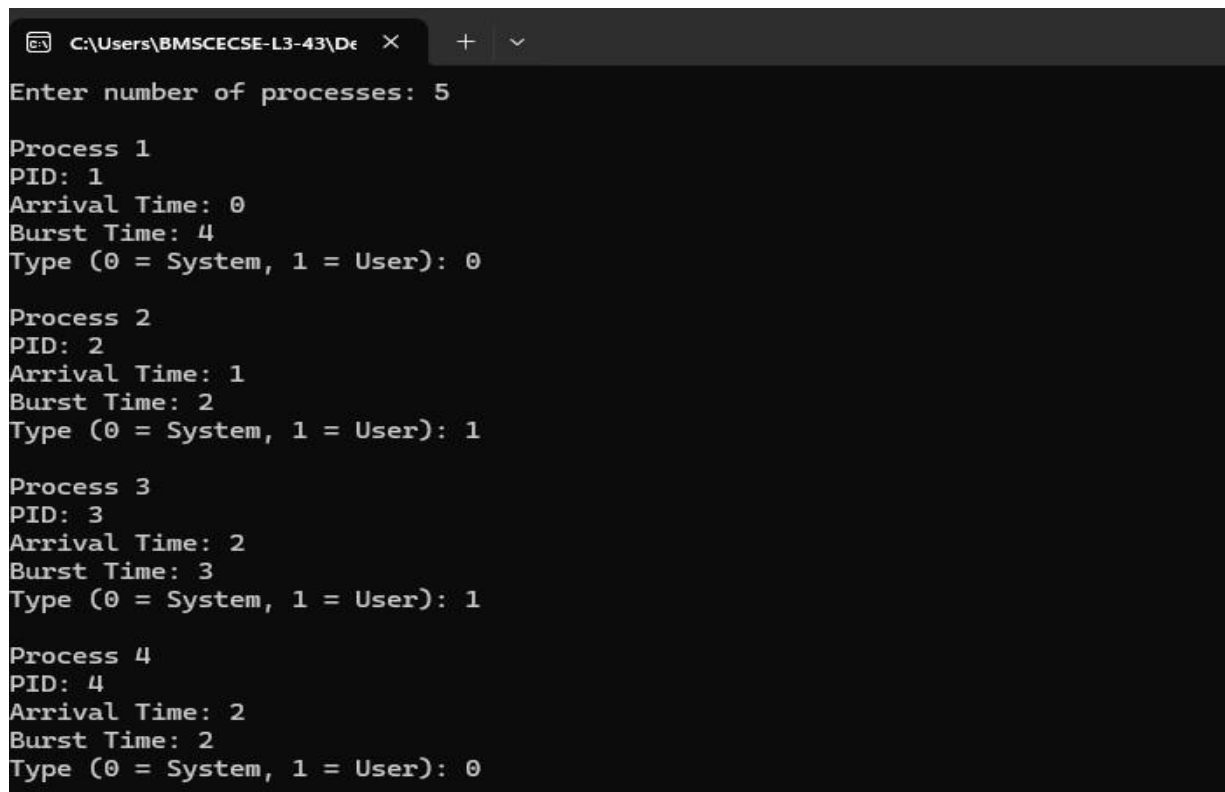
```

        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               PID[i], AT[i], BT[i], TYPE[i],
               CT[i], TAT[i], WT[i]);
    }

    return 0;
}

```

## OUTPUT:



```

C:\Users\BMSCECSE-L3-43\Desktop X + v
Enter number of processes: 5

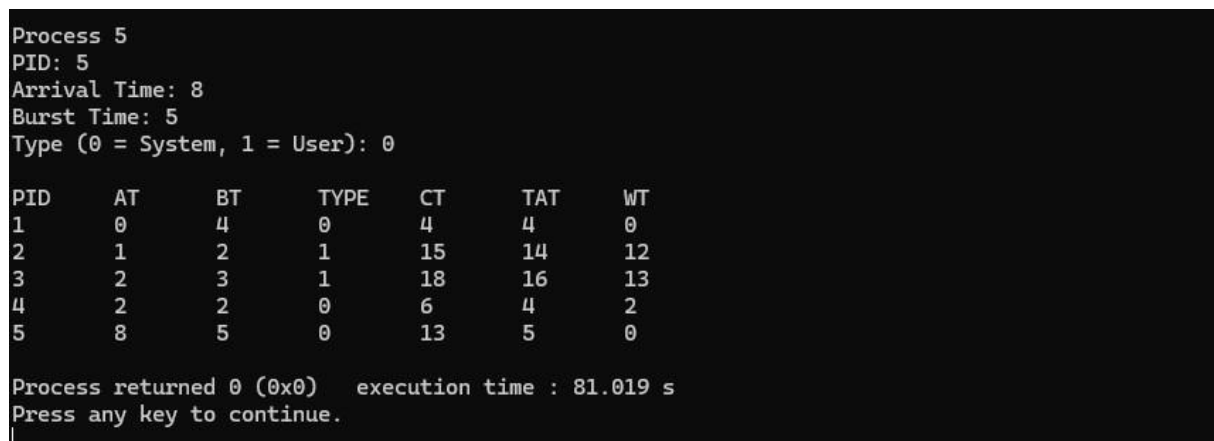
Process 1
PID: 1
Arrival Time: 0
Burst Time: 4
Type (0 = System, 1 = User): 0

Process 2
PID: 2
Arrival Time: 1
Burst Time: 2
Type (0 = System, 1 = User): 1

Process 3
PID: 3
Arrival Time: 2
Burst Time: 3
Type (0 = System, 1 = User): 1

Process 4
PID: 4
Arrival Time: 2
Burst Time: 2
Type (0 = System, 1 = User): 0

```



```

Process 5
PID: 5
Arrival Time: 8
Burst Time: 5
Type (0 = System, 1 = User): 0

PID    AT    BT    TYPE    CT    TAT    WT
1       0     4     0        4     4      0
2       1     2     1       15    14    12
3       2     3     1       18    16    13
4       2     2     0        6     4     2
5       8     5     0       13     5     0

Process returned 0 (0x0)   execution time : 81.019 s
Press any key to continue.

```

3. Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic

```
#include <stdio.h>
#include <math.h>

#define MAX 10

struct Task {
    int id;
    int Ci;
    int Ti;
    int remaining;
};

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

void sortTasks(struct Task tasks[], int n) {
    struct Task temp;

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (tasks[i].Ti > tasks[j].Ti) {
                temp = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = temp;
            }
        }
    }
}

int main() {
```



```

int n, time;
int hyperperiod = 1;
float U = 0.0, bound;

struct Task tasks[MAX];

printf("Enter number of tasks: ");
scanf("%d", &n);

if (n > MAX) {
    printf("Maximum %d tasks allowed.\n", MAX);
    return 0;
}
for (int i = 0; i < n; i++) {
    tasks[i].id = i + 1;

    printf("\nTask %d\n", i + 1);

    printf("Enter Execution Time (Ci): ");
    scanf("%d", &tasks[i].Ci);

    printf("Enter Period (Ti): ");
    scanf("%d", &tasks[i].Ti);

    tasks[i].remaining = 0;

    U += (float)tasks[i].Ci / tasks[i].Ti;
}

printf("\nCPU Utilization = %.4f\n", U);

bound = n * (pow(2.0, 1.0 / n) - 1);
printf("RMS Utilization Bound = %.4f\n", bound);

if (U <= bound)
    printf("System is Schedulable using RMS\n");
else
    printf("System is NOT Guaranteed Schedulable using RMS\n");

sortTasks(tasks, n);

printf("\nPriority Order:\n");
for (int i = 0; i < n; i++) {
    printf("Priority %d -> Task %d (Period = %d)\n",

```

```

        i + 1, tasks[i].id, tasks[i].Ti);
    }

    hyperperiod = tasks[0].Ti;
    for (int i = 1; i < n; i++) {
        hyperperiod = lcm(hyperperiod, tasks[i].Ti);
    }

    printf("\nHyperperiod = %d\n", hyperperiod);

    printf("\nScheduling Timeline:\n");

    for (time = 0; time < hyperperiod; time++) {

        for (int i = 0; i < n; i++) {
            if (time % tasks[i].Ti == 0) {
                tasks[i].remaining = tasks[i].Ci;
            }
        }

        int selected = -1;

        for (int i = 0; i < n; i++) {
            if (tasks[i].remaining > 0) {
                selected = i;
                break;
            }
        }

        if (selected != -1) {
            printf("Time %2d -> Task %d\n", time, tasks[selected].id);
            tasks[selected].remaining--;
        } else {
            printf("Time %2d -> Idle\n", time);
        }
    }

    return 0;
}

```

## OUTPUT:

```
Enter number of tasks: 2

Task 1
Enter Execution Time (Ci): 20
Enter Period (Ti): 50

Task 2
Enter Execution Time (Ci): 35
Enter Period (Ti): 100

CPU Utilization = 0.7500
RMS Utilization Bound = 0.8284
System is Schedulable using RMS

Priority Order:
Priority 1 -> Task 1 (Period = 50)
Priority 2 -> Task 2 (Period = 100)

Hyperperiod = 100

Scheduling Timeline:
Time 0 -> Task 1
Time 1 -> Task 1
Time 2 -> Task 1
Time 3 -> Task 1
Time 4 -> Task 1
Time 5 -> Task 1
Time 6 -> Task 1
Time 7 -> Task 1
Time 8 -> Task 1
```

```
Time 70 -> Task 2
Time 71 -> Task 2
Time 72 -> Task 2
Time 73 -> Task 2
Time 74 -> Task 2
Time 75 -> Idle
Time 76 -> Idle
Time 77 -> Idle
Time 78 -> Idle
Time 79 -> Idle
Time 80 -> Idle
Time 81 -> Idle
Time 82 -> Idle
Time 83 -> Idle
Time 84 -> Idle
Time 85 -> Idle
Time 86 -> Idle
Time 87 -> Idle
Time 88 -> Idle
Time 89 -> Idle
Time 90 -> Idle
Time 91 -> Idle
Time 92 -> Idle
Time 93 -> Idle
Time 94 -> Idle
Time 95 -> Idle
Time 96 -> Idle
Time 97 -> Idle
Time 98 -> Idle
Time 99 -> Idle
```

b) Earliest-deadline First

```
#include <stdio.h>
```

```
#define MAX 10
```

```
typedef struct {  
    int id;  
    int Ci;  
    int Ti;  
    int Di;  
    int remaining;  
    int next_release;  
} Task;
```

```
int gcd(int a, int b) {  
    if (b == 0) return a;  
    return gcd(b, a % b);  
}
```

```
int lcm(int a, int b) {  
    return (a * b) / gcd(a, b);  
}
```

```
int main() {  
    int n;  
    Task t[MAX];  
    int hyperperiod = 1;  
  
    printf("Enter number of tasks: ");  
    scanf("%d", &n);  
  
    for (int i = 0; i < n; i++) {  
        printf("\nTask %d\n", i + 1);  
        t[i].id = i + 1;  
  
        printf("Execution Time (Ci): ");  
        scanf("%d", &t[i].Ci);  
  
        printf("Period (Ti): ");  
        scanf("%d", &t[i].Ti);  
  
        printf("Deadline (Di): ");  
        scanf("%d", &t[i].Di);  
    }
```

```

    t[i].remaining = 0;
    t[i].next_release = 0;

    hyperperiod = lcm(hyperperiod, t[i].Ti);
}

printf("\nHyperperiod (LCM) = %d\n\n", hyperperiod);

for (int time = 0; time < hyperperiod; time++) {

    for (int i = 0; i < n; i++) {
        if (time == t[i].next_release) {
            t[i].remaining += t[i].Ci;
            t[i].next_release += t[i].Ti;
        }
    }

    int selected = -1;
    int min_deadline = 1e9;

    for (int i = 0; i < n; i++) {
        if (t[i].remaining > 0) {
            if (t[i].Di < min_deadline) {
                min_deadline = t[i].Di;
                selected = i;
            }
        }
    }

    if (selected == -1) {
        printf("Time %d: Idle\n", time);
    } else {
        printf("Time %d: Task %d running (Deadline=%d)\n",
            time, t[selected].id, t[selected].Di);

        t[selected].remaining--;
    }
}

return 0;
}

```

## OUTPUT:

```
Enter number of tasks: 3
```

```
Task 1
```

```
Execution Time (Ci): 3
```

```
Period (Ti): 20
```

```
Deadline (Di): 7
```

```
Task 2
```

```
Execution Time (Ci): 2
```

```
Period (Ti): 5
```

```
Deadline (Di): 4
```

```
Task 3
```

```
Execution Time (Ci): 2
```

```
Period (Ti): 10
```

```
Deadline (Di): 8
```

```
Hyperperiod (LCM) = 20
```

```
Time 0: Task 2 running (Deadline=4)
```

```
Time 1: Task 2 running (Deadline=4)
```

```
Time 2: Task 1 running (Deadline=7)
```

```
Time 3: Task 1 running (Deadline=7)
```

```
Time 4: Task 1 running (Deadline=7)
```

```
Time 5: Task 2 running (Deadline=4)
```

```
Time 6: Task 2 running (Deadline=4)
```

```
Time 7: Task 3 running (Deadline=8)
```

```
Time 8: Task 3 running (Deadline=8)
```

```
Time 9: Idle
```

```
Time 10: Task 2 running (Deadline=4)
```

```
Time 11: Task 2 running (Deadline=4)
```

```
Time 12: Task 3 running (Deadline=8)
```

```
Time 13: Task 3 running (Deadline=8)
```

```
Time 14: Idle
```

```
Time 15: Task 2 running (Deadline=4)
```

```
Time 16: Task 2 running (Deadline=4)
```

```
Time 17: Idle
```

```
Time 18: Idle
```

```
Time 19: Idle
```

```
Process returned 0 (0x0)    execution time : 24.574 s
```

```
Press any key to continue.
```

c) Proportional-sharing

```
#include <stdio.h>

struct Task {
    int id, execution, period;
    float share;
};

int main() {
    int n, i;
    struct Task t[10];

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        t[i].id = i + 1;
        printf("Enter execution time and period for Task %d: ", i+1);
        scanf("%d %d", &t[i].execution, &t[i].period);

        t[i].share = (float)t[i].execution / t[i].period;
    }

    printf("\nProportional Scheduling:\n");
    for(i = 0; i < n; i++) {
        printf("Task %d gets %.2f CPU share\n", t[i].id, t[i].share);
    }

    return 0;
}
```

### OUTPUT:

```
Enter number of tasks: 3
Enter execution time and period for Task 1: 1
4
Enter execution time and period for Task 2: 2
5
Enter execution time and period for Task 3: 1
6

Proportional Scheduling:
Task 1 gets 0.25 CPU share
Task 2 gets 0.40 CPU share
Task 3 gets 0.17 CPU share

Process returned 0 (0x0)   execution time : 12.718 s
Press any key to continue.
|
```



4. Write a C program to simulate:

a) Producer-Consumer problem using semaphores.

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1;
int full = 0;
int empty = 3;
int x = 0;

void wait() {
    mutex--;
}

void signal() {
    mutex++;
}

void producer() {
    if ((mutex == 1) && (empty != 0)) {
        wait();
        full++;
        empty--;
        x++;
        printf("Producer has produced: Item %d\n", x);
        signal();
    } else {
        printf("Buffer is full!\n");
    }
}

void consumer() {
    if ((mutex == 1) && (full != 0)) {
        wait();
        full--;
        empty++;
        printf("Consumer has consumed: Item %d\n", x);
        x--;
        signal();
    } else {
        printf("Buffer is empty!\n");
    }
}
```

```

}

int main() {
    int choice;

    while (1) {
        printf("\nEnter 1.Producer 2.Consumer 3.Exit\n");
        printf("Enter your choice:\n");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                producer();
                break;
            case 2:
                consumer();
                break;
            case 3:
                exit(0);
                break;
            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}

```

### OUTPUT:

```
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
1
Producer has produced: Item 1

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
1
Producer has produced: Item 2

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
1
Producer has produced: Item 3

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
1
Buffer is full!
```

```
Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
2
Consumer has consumed: Item 3

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
2
Consumer has consumed: Item 2

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
2
Consumer has consumed: Item 1

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
2
Buffer is empty!

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
3
```

b) Dining-Philosopher's problem

```
#include <stdio.h>

#define MAX 10

int n;
int hungry;
int pos[MAX];

int canEat(int p, int eating[]) {
    int left = (p + n - 1) % n;
    int right = (p + 1) % n;

    if (eating[left] == 0 && eating[right] == 0)
        return 1;
    return 0;
}

void oneAtATime() {
    int eating[MAX] = {0};

    printf("Allow one philosopher to eat at any time\n");

    for (int i = 0; i < hungry; i++) {
        int p = pos[i];

        for (int j = 0; j < hungry; j++) {
            printf("P %d is waiting\n", pos[j]);
        }

        if (canEat(p, eating)) {
            eating[p] = 1;
            printf("P %d is granted to eat\n", p);
            printf("P %d has finished eating\n", p);
            eating[p] = 0;
        }
    }
}

void twoAtATime() {
    int eating[MAX] = {0};

    printf("Allow two philosophers to eat at any time\n");
```

```

for (int i = 0; i < hungry; i++) {
    int p = pos[i];

    if (canEat(p, eating)) {
        eating[p] = 1;
        printf("P %d is granted to eat\n", p);
    } else {
        printf("P %d is waiting\n", p);
    }
}

for (int i = 0; i < hungry; i++) {
    int p = pos[i];
    if (eating[p]) {
        printf("P %d has finished eating\n", p);
        eating[p] = 0;
    }
}
}

int main() {
    int choice;

    printf("Enter the total number of philosophers: ");
    scanf("%d", &n);

    printf("How many are hungry: ");
    scanf("%d", &hungry);

    for (int i = 0; i < hungry; i++) {
        printf("Enter philosopher %d position (1 to %d): ", i + 1, n);
        scanf("%d", &pos[i]);
        pos[i]--;
    }

    while (1) {
        printf("\n1. One can eat at a time\n");
        printf("2. Two can eat at a time\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:

```

```

        oneAtATime();
        break;
    case 2:
        twoAtATime();
        break;
    case 3:
        return 0;
    default:
        printf("Invalid choice\n");
    }
}

return 0;
}

```

### **OUTPUT:**

```

How many are hungry: 3
Enter philosopher 1 position (1 to 5): 2
Enter philosopher 2 position (1 to 5): 4
Enter philosopher 3 position (1 to 5): 5

1. One can eat at a time
2. Two can eat at a time
3. Exit
Enter your choice: 1
Allow one philosopher to eat at any time
P 1 is waiting
P 3 is waiting
P 4 is waiting
P 1 is granted to eat
P 1 has finished eating
P 1 is waiting
P 3 is waiting
P 4 is waiting
P 3 is granted to eat
P 3 has finished eating
P 1 is waiting
P 3 is waiting
P 4 is waiting
P 4 is granted to eat
P 4 has finished eating

1. One can eat at a time
2. Two can eat at a time
3. Exit
Enter your choice: 3

```

5. Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>

int main()
{
    int n, m;

    printf("Enter number of processes -- ");
    scanf("%d", &n);

    printf("Enter number of resources -- ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];
    int finish[n], safe[n];

    for (int i = 0; i < n; i++)
    {
        printf("\nEnter details for P%d\n", i);

        printf("Enter allocation -- ");
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }

        printf("Enter Max -- ");
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }
    printf("\nEnter Available Resources -- ");
    for (int i = 0; i < m; i++)
    {
        scanf("%d", &avail[i]);
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
```

```

        {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    int pid;
    int request[m];

    printf("\nEnter New Request Details --\n");

    printf("Enter pid -- ");
    scanf("%d", &pid);

    printf("Enter Request for Resources -- ");
    for (int i = 0; i < m; i++)
    {
        scanf("%d", &request[i]);
    }

    int possible = 1;

    for (int i = 0; i < m; i++)
    {
        if (request[i] > need[pid][i] || request[i] > avail[i])
        {
            possible = 0;
            break;
        }
    }

    if (!possible)
    {
        printf("\nRequest cannot be granted.\n");
        return 0;
    }

    for (int i = 0; i < m; i++)
    {
        avail[i] -= request[i];
        alloc[pid][i] += request[i];
        need[pid][i] -= request[i];
    }

    for (int i = 0; i < n; i++)

```



```

{
    finish[i] = 0;
}

int count = 0;

while (count < n)
{
    int found = 0;

    for (int i = 0; i < n; i++)
    {
        if (finish[i] == 0)
        {
            int flag = 1;

            for (int j = 0; j < m; j++)
            {
                if (need[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }

            if (flag)
            {
                for (int j = 0; j < m; j++)
                {
                    avail[j] += alloc[i][j];
                }

                printf("\nP%d is visited(", i);

                for (int j = 0; j < m; j++)
                {
                    printf("%d", avail[j]);

                    if (j != m - 1)
                        printf(" ");
                }

                printf(")");
            }
        }
    }

    count++;
}

```

```

        safe[count] = i;
        count++;

        finish[i] = 1;
        found = 1;
    }
}

if (found == 0)
{
    printf("\nSYSTEM IS NOT SAFE STATE\n");
    return 0;
}

printf("\n\nSYSTEM IS IN SAFE STATE\n");

printf("The Safe Sequence is -- (");

for (int i = 0; i < n; i++)
{
    printf("P%d", safe[i]);

    if (i != n - 1)
        printf(" ");
}

printf("\n");

return 0;
}

```

### OUTPUT:

```
Enter number of processes -- 5
Enter number of resources -- 3
```

```
Enter details for P0
Enter allocation -- 0 1 0
Enter Max -- 7 5 3
```

```
Enter details for P1
Enter allocation -- 2 0 0
Enter Max -- 3 2 2
```

```
Enter details for P2
Enter allocation -- 3 0 2
Enter Max -- 9 0 2
```

```
Enter details for P3
Enter allocation -- 2 1 1
Enter Max -- 2 2 2
```

```
Enter details for P4
Enter allocation -- 0 0 2
Enter Max -- 4 3 3
```

```
Enter Available Resources -- 3 3 2
```

```
Enter New Request Details --
Enter pid -- 1
Enter Request for Resources -- 1 0 2
```

```
P1 is visited(5 3 2)
P3 is visited(7 4 3)
P4 is visited(7 4 5)
P0 is visited(7 5 5)
P2 is visited(10 5 7)
```

```
SYSTEM IS IN SAFE STATE
The Safe Sequence is -- (P1 P3 P4 P0 P2)
```

```
Process returned 0 (0x0)    execution time : 79.947 s
Press any key to continue.
```

## b) Deadlock Detection

```
#include <stdio.h>

int main()
{
    int n, m;

    printf("Enter no of processes: ");
    scanf("%d", &n);

    printf("Enter no of resources: ");
    scanf("%d", &m);

    int alloc[n][m], req[n][m], avail[m], work[m];
    int finish[n], seq[n];

    for (int i = 0; i < n; i++)
    {
        printf("\nP%d Allocation: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

        printf("P%d Request: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &req[i][j]);
    }

    printf("\nAvailable Resources: ");
    for (int i = 0; i < m; i++)
    {
        scanf("%d", &avail[i]);
        work[i] = avail[i];
    }

    for (int i = 0; i < n; i++)
    {
        int sum = 0;
        finish[i] = 0;

        for (int j = 0; j < m; j++)
            sum += alloc[i][j];

        if (sum == 0)
```

```

        finish[i] = 1;
    }

    int k = 0, found;

    do
    {
        found = 0;

        for (int i = 0; i < n; i++)
        {
            if (!finish[i])
            {
                int flag = 1;

                for (int j = 0; j < m; j++)
                {
                    if (req[i][j] > work[j])
                    {
                        flag = 0;
                        break;
                    }
                }

                if (flag)
                {
                    for (int j = 0; j < m; j++)
                        work[j] += alloc[i][j];

                    finish[i] = 1;
                    seq[k++] = i;
                    found = 1;

                    printf("P%d visited\n", i);
                }
            }
        }
    } while (found);

    int deadlock = 0;

    for (int i = 0; i < n; i++)
    {

```

```

    if (!finish[i])
    {
        deadlock = 1;
        break;
    }
}

if (deadlock)
{
    printf("\nSystem is in Deadlock\n");

    printf("Deadlocked Processes: ");
    for (int i = 0; i < n; i++)
        if (!finish[i])
            printf("P%d ", i);
}
else
{
    printf("\nSystem is not in Deadlock\n");

    printf("Sequence: ");
    for (int i = 0; i < k; i++)
        printf("P%d ", seq[i]);
}

return 0;
}

```

### **OUTPUT:**

```

Enter no of processes: 5
Enter no of resources: 3

P0 Allocation: 0 1 0
P0 Request: 0 0 0

P1 Allocation: 2 0 0
P1 Request: 2 0 2

P2 Allocation: 3 0 3
P2 Request: 0 0 1

P3 Allocation: 2 1 1
P3 Request: 1 0 0

P4 Allocation: 0 0 2
P4 Request: 0 0 2

Available Resources: 0 0 0
P0 visited

System is in Deadlock
Deadlocked Processes: P1 P2 P3 P4

```

6. Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>
```

```
void firstFit(int blockSize[], int m, int processSize[], int n) {  
    int allocation[n];
```

```
    for (int i = 0; i < n; i++)  
        allocation[i] = -1;
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {
```

```
            if (blockSize[j] >= processSize[i]) {  
                allocation[i] = j;
```

```
                blockSize[j] = -1;  
                break;
```

```
            }  
        }  
    }
```

```
    printf("\nFIRST FIT Allocation\n");  
    printf("Process No.\tProcess Size\tBlock No.\n");
```

```
    for (int i = 0; i < n; i++) {  
        printf("%d\t%d\t", i + 1, processSize[i]);
```

```
        if (allocation[i] != -1)  
            printf("%d\n", allocation[i] + 1);
```

```
        else  
            printf("Not Allocated\n");
```

```
    }  
}
```

```
void bestFit(int blockSize[], int m, int processSize[], int n) {  
    int allocation[n];
```

```
    for (int i = 0; i < n; i++)  
        allocation[i] = -1;
```

```
    for (int i = 0; i < n; i++) {  
        int bestIdx = -1;
```

```

    for (int j = 0; j < m; j++) {

        if (blockSize[j] >= processSize[i]) {

            if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                bestIdx = j;
            }
        }
    }

    if (bestIdx != -1) {
        allocation[i] = bestIdx;

        blockSize[bestIdx] = -1;
    }
}

printf("\nBEST FIT Allocation\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;

        for (int j = 0; j < m; j++) {

            if (blockSize[j] >= processSize[i]) {

```



```

        if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
            worstIdx = j;
        }
    }
}

if (worstIdx != -1) {
    allocation[i] = worstIdx;

    blockSize[worstIdx] = -1;
}
}

printf("\nWORST FIT Allocation\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m], blockSize1[m], blockSize2[m];

    printf("Enter sizes of memory blocks:\n");

    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);

        blockSize1[i] = blockSize[i];
        blockSize2[i] = blockSize[i];
    }

    printf("Enter number of processes: ");

```

```

scanf("%d", &n);

int processSize[n];

printf("Enter sizes of processes:\n");

for (int i = 0; i < n; i++) {
    scanf("%d", &processSize[i]);
}

firstFit(blockSize, m, processSize, n);
bestFit(blockSize1, m, processSize, n);
worstFit(blockSize2, m, processSize, n);

return 0;
}

```

### **OUTPUT:**

```

Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

FIRST FIT Allocation
Process No.    Process Size    Block No.
1              212           2
2              417           5
3              112           3
4              426          Not Allocated

BEST FIT Allocation
Process No.    Process Size    Block No.
1              212           4
2              417           2
3              112           3
4              426           5

WORST FIT Allocation
Process No.    Process Size    Block No.
1              212           5
2              417           2
3              112           4
4              426          Not Allocated

```

7. Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal

```
#include <stdio.h>
```

```
int search(int key, int frame[], int f) {  
    for (int i = 0; i < f; i++) {  
        if (frame[i] == key)  
            return 1;  
    }  
    return 0;  
}
```

```
void fifo(int pages[], int n, int f) {  
    int frame[f];  
    int index = 0, faults = 0;  
  
    for (int i = 0; i < f; i++)  
        frame[i] = -1;  
  
    printf("\nFIFO Page Replacement Process:\n");  
  
    for (int i = 0; i < n; i++) {  
  
        if (!search(pages[i], frame, f)) {  
  
            frame[index] = pages[i];  
            index = (index + 1) % f;  
            faults++;  
  
            printf("PF No. %d: ", faults);  
  
            for (int j = 0; j < f; j++) {  
                if (frame[j] != -1)  
                    printf("%d ", frame[j]);  
                else  
                    printf("- ");  
            }  
            printf("\n");  
        }  
    }  
  
    printf("FIFO Page Faults: %d\n", faults);  
}
```

```

void lru(int pages[], int n, int f) {
    int frame[f], time[f];
    int faults = 0, count = 0;

    for (int i = 0; i < f; i++) {
        frame[i] = -1;
        time[i] = 0;
    }

    printf("\nLRU Page Replacement Process:\n");

    for (int i = 0; i < n; i++) {

        int found = 0;

        for (int j = 0; j < f; j++) {
            if (frame[j] == pages[i]) {
                count++;
                time[j] = count;
                found = 1;
                break;
            }
        }

        if (!found) {

            int pos = 0;

            for (int j = 1; j < f; j++) {
                if (time[j] < time[pos])
                    pos = j;
            }

            frame[pos] = pages[i];
            count++;
            time[pos] = count;
            faults++;

            printf("PF No. %d: ", faults);

            for (int j = 0; j < f; j++) {
                if (frame[j] != -1)
                    printf("%d ", frame[j]);
                else

```

```

        printf("- ");
    }
    printf("\n");
}

printf("LRU Page Faults: %d\n", faults);
}

void optimal(int pages[], int n, int f) {
    int frame[f];
    int faults = 0;

    for (int i = 0; i < f; i++)
        frame[i] = -1;

    printf("\nOptimal Page Replacement Process:\n");

    for (int i = 0; i < n; i++) {

        if (!search(pages[i], frame, f)) {

            int pos = -1, farthest = i + 1;

            for (int j = 0; j < f; j++) {
                if (frame[j] == -1) {
                    pos = j;
                    break;
                }
            }

            if (pos == -1) {

                int max = -1;

                for (int j = 0; j < f; j++) {

                    int k;

                    for (k = i + 1; k < n; k++) {
                        if (frame[j] == pages[k])
                            break;
                    }
                }
            }
        }
    }
}

```

```

        if (k > max) {
            max = k;
            pos = j;
        }
    }
}

frame[pos] = pages[i];
faults++;

printf("PF No. %d: ", faults);

for (int j = 0; j < f; j++) {
    if (frame[j] != -1)
        printf("%d ", frame[j]);
    else
        printf("- ");
}
printf("\n");
}

printf("Optimal Page Faults: %d\n", faults);
}

int main() {

    int f, n;

    printf("Enter the number of Frames: ");
    scanf("%d", &f);

    printf("Enter the length of reference string: ");
    scanf("%d", &n);

    int pages[n];

    printf("Enter the reference string: ");

    for (int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    fifo(pages, n, f);
    lru(pages, n, f);

```

```
    optimal(pages, n, f);  
  
    return 0;  
}
```

### **OUTPUT:**

```
Enter the number of Frames: 3  
Enter the length of reference string: 6  
Enter the reference string: 1 4 2 3 2 1  
FIFO Page Replacement Process:  
PF No. 1: 1 - -  
PF No. 2: 1 4 -  
PF No. 3: 1 4 2  
PF No. 4: 3 4 2  
PF No. 5: 3 1 2  
FIFO Page Faults: 5  
  
LRU Page Replacement Process:  
PF No. 1: 1 - -  
PF No. 2: 1 4 -  
PF No. 3: 1 4 2  
PF No. 4: 3 4 2  
PF No. 5: 3 1 2  
LRU Page Faults: 5  
  
Optimal Page Replacement Process:  
PF No. 1: 1 - -  
PF No. 2: 1 4 -  
PF No. 3: 1 2 -  
PF No. 4: 1 2 3  
Optimal Page Faults: 4
```